

Complex Numbers and AC Circuits

Engineering Computation with Python – Chapter 8

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

1 Rectangular and Polar Form

2 Impedance

3 Parallel RLC Circuits

4 Power in AC Circuits

5 Resonance

6 Summary

1 Rectangular and Polar Form

2 Impedance

3 Parallel RLC Circuits

4 Power in AC Circuits

5 Resonance

6 Summary

DC Needs Real Numbers. AC Needs One More Idea.

Your circuits course starts with **DC**: current flows one way, Ohm's law is simple, $V = IR$.

Real power grids, audio equipment, and wireless devices run on **AC**: voltage and current oscillate, and components shift their **timing (phase)** relative to each other.

The brilliant shortcut

Treat AC signals as **complex numbers**. The exact same simple algebra used for DC circuits then works for AC — no trigonometric identities required.

Python supports complex numbers natively. No extra library needed.

Rectangular form — real and imaginary parts separately:

$$Z = a + jb.$$

Polar form — magnitude and direction:

$$Z = |Z|\angle\theta, \quad |Z| = \sqrt{a^2 + b^2}, \quad \theta = \arctan\left(\frac{b}{a}\right).$$

Going back the other way uses trigonometry:

$$a = |Z| \cos \theta, \quad b = |Z| \sin \theta.$$

Rectangular tells you resistance and reactance separately; polar tells you the overall opposition and the timing shift.

Converting Between Forms in Python

```
import numpy as np

Z = 3 + 4j                                # rectangular form

# Rectangular -> polar
magnitude = abs(Z)
angle_deg = np.degrees(np.angle(Z))
print(f"|Z| = {magnitude:.2f}, angle = {angle_deg:.2f} degrees")

# Polar -> rectangular
a = magnitude * np.cos(np.radians(angle_deg))
b = magnitude * np.sin(np.radians(angle_deg))
print(f"back to rectangular: {a:.2f} + {b:.2f}j")
```

|Z| = 5.00, angle = 53.13 degrees
back to rectangular: 3.00 + 4.00j

A resistance of $3\ \Omega$ and a reactance of $4\ \Omega$ combine to a total opposition of $5\ \Omega$ — **not** $7\ \Omega$.

Why they don't simply add

Reactance and resistance act a **quarter-cycle apart** in time. That timing offset is precisely what the imaginary unit encodes — it is the familiar right-triangle relationship in disguise.

1 Rectangular and Polar Form

2 **Impedance**

3 Parallel RLC Circuits

4 Power in AC Circuits

5 Resonance

6 Summary

Impedance: The AC Generalization of Resistance

In DC circuits, the relevant quantity is **resistance** (R). In AC circuits, this expands to **impedance** (Z):

$$Z = R + jX.$$

Real part: R

Opposition to current that dissipates energy as heat.
Independent of frequency.

Imaginary part: X (reactance)

Opposition from magnetic (inductor) or electric (capacitor) fields. Stores and releases energy – no dissipation, just a timing shift.

Mathematics uses $i = \sqrt{-1}$; electrical engineering uses j , because i is already current. Python follows the EE convention.

Reactance Depends on Frequency

At angular frequency $\omega = 2\pi f$:

Inductors (L)

Oppose **rapid changes in current**.

$$X_L = \omega L, \quad +jX_L.$$

Reactance **grows** with frequency.

Capacitors (C)

Oppose **rapid changes in voltage**.

$$X_C = \frac{1}{\omega C}, \quad -jX_C.$$

Reactance **shrinks** with frequency.

```
# Defining complex numbers directly
Z1 = 40 + 30j      # 40 ohms real, +30 ohms reactance
Z2 = 20 - 15j      # 20 ohms real, -15 ohms reactance

print(type(Z1))    # Output: <class 'complex'>
```

Watch out

A number must come immediately before j (3j, 1j). Writing just j alone is an ordinary variable name — a `NameError` if undefined.

```
import numpy as np

Z = 40 + 22.35j

print("Real part (R):", Z.real)          # 40.0
print("Imaginary part (X):", Z.imag)     # 22.35

magnitude = abs(Z)
print(f"Total opposition: {magnitude:.2f} Ohms") # 45.82

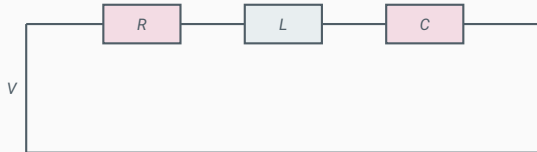
phase_degrees = np.degrees(np.angle(Z))
print(f"Phase shift: {phase_degrees:.1f} deg")   # 29.2
```

$R = 40\ \Omega$, $L = 200\ \text{mH}$, $C = 50\ \mu\text{F}$, $f = 60\ \text{Hz}$.

Series rule

The same current flows through every component, so impedances **add**:

$$Z = Z_R + Z_L + Z_C.$$



Solving the Series Circuit in Python

```
import numpy as np

R, L, C, f = 40.0, 0.2, 50e-6, 60.0
omega = 2 * np.pi * f

Z_R = R
Z_L = 1j * omega * L
Z_C = -1j / (omega * C)

Z_total = Z_R + Z_L + Z_C    # series: impedances just add!

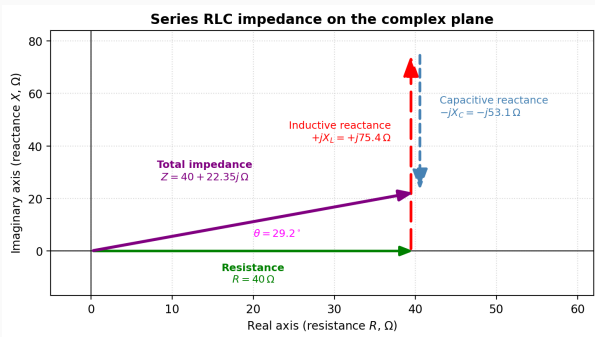
print(f"Total Impedance:    {Z_total.real:.1f} + {Z_total.imag:.2f}j 0hms")
print(f"Magnitude:         {np.abs(Z_total):.2f} 0hms")
print(f"Phase Angle:        {np.degrees(np.angle(Z_total)):.1f} degrees")
```

Total Impedance: $40.0 + 22.35j$ Ohms

Magnitude: 45.82 Ohms Phase Angle: 29.2 degrees

Interpret

Without a single trigonometry problem: the circuit opposes current with magnitude 45.82Ω , and current **lags** voltage by 29.2° — a positive angle means **inductive reactance dominates** at this frequency.



Start at the origin: step 40Ω right (R), step up 75.4Ω ($+jX_L$), step down 53.1Ω ($-jX_C$).

The purple vector from origin to $40 + 22.35j$ is the total impedance Z : its **length** is the magnitude, its **angle** is the phase.

The vector sum shows visually how inductors and capacitors actively oppose each other's effects.

A preview of resonance

If the frequency were tuned so that $X_L = X_C$, the upward and downward reactance steps would be exactly equal, and the total impedance would lie flat on the real axis.

This fundamental electrical phenomenon is **resonance** — covered later in this chapter.

1 Rectangular and Polar Form

2 Impedance

3 **Parallel RLC Circuits**

4 Power in AC Circuits

5 Resonance

6 Summary

Parallel Impedances Combine as Reciprocals

In a series circuit the impedances add directly. In a **parallel** circuit they combine as reciprocals, exactly as parallel resistances do in DC:

$$\frac{1}{Z_{\text{total}}} = \frac{1}{Z_R} + \frac{1}{Z_L} + \frac{1}{Z_C}.$$

No manual algebra needed

This looks troublesome by hand — rationalizing denominators, separating real and imaginary parts. Python's complex division handles it in **one line**.

Same Components, Connected in Parallel

```
import numpy as np

R, L, C, f = 40.0, 0.2, 50e-6, 60.0
omega = 2 * np.pi * f

Z_R = R
Z_L = 1j * omega * L
Z_C = -1j / (omega * C)

# Parallel combination: reciprocal of the sum of reciprocals
Z_total = 1 / (1/Z_R + 1/Z_L + 1/Z_C)

print(f"Z_total = {Z_total.real:.2f} + {Z_total.imag:.2f}j Ohms")
print(f"Magnitude: {np.abs(Z_total):.2f} Ohms")
print(f"Phase angle: {np.degrees(np.angle(Z_total)):.2f} degrees")
```

$Z_{\text{total}} = 38.10 + -8.51j \text{ Ohms}$ Magnitude: 39.04 Phase: -12.60°

Series (same parts)

$40 + 22.35j \Omega$, magnitude 45.82Ω , phase $+29.2^\circ$ – net **inductive**.

Parallel (same parts)

$38.10 - 8.51j \Omega$, magnitude 39.04Ω , phase -12.6° – net **capacitive**.

Same three components. Different connection. Opposite character.

1 Rectangular and Polar Form

2 Impedance

3 Parallel RLC Circuits

4 **Power in AC Circuits**

5 Resonance

6 Summary

In DC, power is simply $P = VI$. In AC, that formula fails whenever the load is not purely resistive – voltage and current no longer peak at the same instant.

Three separate quantities are needed, and confusing them is a classic error:

- **Real power** P (W) – actually converted to heat or work. The only part the meter charges for.
- **Reactive power** Q (VAR) – sloshes back and forth, doing no net work.
- **Apparent power** S (VA) – what the cabling must be sized to carry.

$$S = \sqrt{P^2 + Q^2}, \quad P = S \cos \theta, \quad Q = S \sin \theta.$$

$$\text{power factor} = \cos \theta = \frac{P}{S}.$$

What power factor tells you

What fraction of delivered capacity does useful work. Purely resistive: $\cos \theta = 1$. Purely reactive: $\cos \theta = 0$ — no work at all.

```
import numpy as np

R, L, C, f = 40.0, 0.2, 50e-6, 60.0
V = 120.0                                # supply voltage (RMS)

omega = 2 * np.pi * f
Z = R + 1j*omega*L - 1j/(omega*C)

I = V / Z                                # complex current (Ohm's law)
S = V * np.conj(I)                        # complex power

print(f"Real power P:      {S.real:.2f} W")
print(f"Reactive power Q:  {S.imag:.2f} VAR")
print(f"Apparent power S:   {np.abs(S):.2f} VA")
print(f"Power factor:       {np.cos(np.angle(Z)):.4f}")
```

P: 274.37 W Q: 153.28 VAR S: 314.28 VA PF: 0.8730

Check

P must equal $|I|^2 R = 2.619^2 \times 40 = 274.4 \text{ W}$ – it does. And $\sqrt{P^2 + Q^2} = 314.3 \text{ VA}$ matches S .

Interpret

Only 274 W of the 314 VA supplied does useful work. Industrial customers are penalised for poor power factor – capacitors are added alongside motors to cancel inductive Q and pull $\cos \theta$ toward unity.

1 Rectangular and Polar Form

2 Impedance

3 Parallel RLC Circuits

4 Power in AC Circuits

5 **Resonance**

6 Summary

$X_L = \omega L$ grows with frequency; $X_C = 1/\omega C$ shrinks. At some frequency the two must be equal.

Resonant frequency

$$\omega_0 L = \frac{1}{\omega_0 C} \quad \Rightarrow \quad f_0 = \frac{1}{2\pi\sqrt{LC}}.$$

At resonance

The imaginary part of Z vanishes: $Z = R$. The circuit behaves as though the inductor and capacitor were not there.

```
import numpy as np

R, L, C, V = 40.0, 0.2, 50e-6, 120.0

f0 = 1 / (2*np.pi*np.sqrt(L*C))
omega0 = 2*np.pi*f0

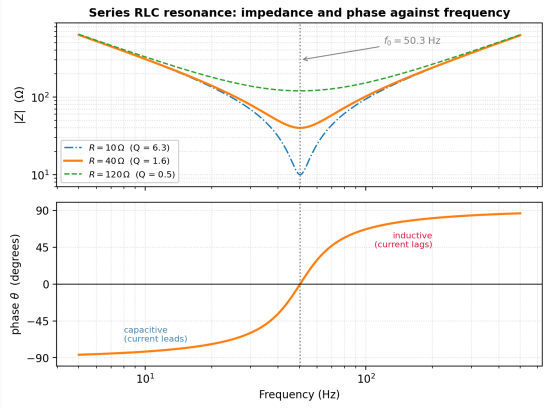
X_L = omega0 * L
X_C = 1 / (omega0 * C)
Z0 = R + 1j*X_L - 1j*X_C

print(f"Resonant frequency: {f0:.2f} Hz")
print(f"Z at resonance: {Z0.real:.1f} + {Z0.imag:.4f}j Ohms")
print(f"Current at f0: {np.abs(V/Z0):.3f} A")
```

$$f_0 = 50.33 \text{ Hz} \quad Z = 40.0 + 0.0000j \quad \text{Current} = 3.000 \text{ A}$$

- **Current peaks.** Impedance is smallest, so a modest voltage drives a large current — exploited in radio tuning, dangerous in power systems.
- **Power factor becomes unity.** $\theta = 0$, $\cos \theta = 1$: every VA does useful work.
- **Character flips with frequency.** Below f_0 : capacitive, current leads. Above f_0 : inductive, current lags.

Check: the two reactances agree to three decimals, but Z 's imaginary part is never exactly zero in floating point — a reason to avoid testing with `==`.



Both axes logarithmic. The minimum sits at f_0 for every curve — changing R moves the dip up or down, never sideways.

How sharp the resonance dip is depends on resistance:

$$Q_{\text{factor}} = \frac{1}{R} \sqrt{\frac{L}{C}}.$$

Low R (10Ω)

Deep, narrow dip. Strong but selective response — good for tuning a radio to one station.

High R (120Ω)

Shallow, broad dip. Still resonates, but barely distinguishes f_0 from its neighbours.

This Q (dimensionless sharpness) is unrelated to the reactive power Q from the previous section — an unfortunate clash of notation in electrical engineering.

1 Rectangular and Polar Form

2 Impedance

3 Parallel RLC Circuits

4 Power in AC Circuits

5 Resonance

6 Summary

- Python writes complex numbers with a `j` suffix ($3 + 4j$); must be `1j`, never bare `j`.
- **Rectangular** suits $+/-$; **polar** suits $\times/\div$. `abs()` and `np.angle()` convert between them.
- **Impedance** $Z = R + jX$ generalises resistance; $X_L = \omega L$ (positive), $X_C = -1/\omega C$ (negative).
- Series impedances **add**; parallel impedances combine by **reciprocal** – same rule as DC resistors, complex arithmetic.
- **Magnitude** sets current flow; **phase** says whether the load is capacitive or inductive.
- AC power splits into P (W), Q (VAR), S (VA), with $S = \sqrt{P^2 + Q^2}$ and complex power $S = VI^*$.
- **Power factor** $\cos \theta = P/S$; correcting it (adding capacitance to inductive loads) reduces current for the same real power.
- At **resonance**, $f_0 = 1/(2\pi\sqrt{LC})$, reactances cancel, $Z = R$, current peaks, power factor hits unity.
- **Quality factor** $(1/R)\sqrt{L/C}$ measures how sharply a circuit selects its resonant frequency.

This chapter turned AC circuit analysis into ordinary algebra with complex numbers. The next chapter extends this to circuits with **multiple** loops and nodes.

- Chapter 9: Matrices and Circuit Equations — representing a whole circuit as $A\mathbf{x} = \mathbf{b}$ and solving it in one line.
- The complex-number skills here carry over directly: AC mesh equations are solved with the same numpy linear algebra, just with complex entries.

Next: Matrices and Circuit Equations